ORBISA

OFFICE OF THE CTO · INTERNAL ENGINEERING BRIEF

# Orbisa — Engineering Case Study

Trustworthy context for AI agents: making Helix observable

**Platform & AI Engineering · October 2026**

Orbisa, Brisklane and Helix are fictional.

| DOCUMENT CONTROL | |
|---|---|
| Document | Engineering case study for the Platform & AI Engineering team |
| Owner | **Daniel Okafor**, CTO |
| Stakeholders | Tomás Herrera (Head of RevOps) · Aisha Bello (VP Customer Success) · Ken Watanabe (Support Lead) |
| Status | **Helix v0 in pilot, observability not started** |
| Date | October 2026 |
| Repository | `ws-5-langfuse-observability` (paths in this document are relative to the repo root) |
| Classification | Internal. Contains customer names and contract terms. |

Sources · Context · Agent · Observability · Langfuse

# Contents




| | | |
|---|---|---|
| 01 | Executive summary | 3 |
| 02 | Company background<br>Products, customers, plans, people, the Brisklane acquisition | 4 |
| 03 | The business problem<br>How teams find answers today, the Orion incident, other symptoms, the risk | 5 |
| 04 | Problem analysis<br>Four failure modes and why search and plain RAG fail on them | 7 |
| 05 | Requirements<br>Functional (FR) and non-functional (NFR), with Helix v0 status | 8 |
| 06 | Solution overview<br>Helix and Ask Orbisa; how one question flows | 9 |
| 07 | Architecture<br>Diagram, components, request sequence for Q01, temporal model | 10 |
| 08 | Technology stack | 12 |
| 09 | Tool selection<br>Eight decision records, including the Langfuse deployment matrix | 13 |
| 10 | Current state (brownfield)<br>What exists, what works, known gaps | 15 |
| 11 | The mission<br>Questions, success criteria, constraints, out of scope | 16 |
| A | Systems inventory | 18 |
| B | People and customers | 18 |
| C | Ground-truth overview | 19 |
| D | Glossary | 19 |


**HOW TO USE THIS BRIEF**

Read it as a team. Sections 1–4 explain the business and the problem. Sections 5–10 describe what we already built and why. Section 11 is your mission. Discuss each section together, then open the repository and check the claims against the code and the data.

Every company fact in this document comes from the story bible, `data/orbisa/README.md`. Quotes come from the exports in `data/orbisa/`. If you find a conflict, the story bible wins.

# 01 Executive summary

ONE PAGE

Orbisa's knowledge lives in seven systems. Policies change, but old pages stay online. People answer customers from the wrong page, and nobody notices until a customer pushes back.

## The problem

On 2026-09-29 Orion Freight Systems, an Enterprise customer worth USD 186,000 a year, asked for a refund. A support agent replied that refunds are only possible within 30 days of renewal. That rule is from 2023. It was replaced on 2026-03-01, and Orion's signed contract has its own refund clause (7.3). The answer was wrong on the rule, wrong on the contract, and it named no approver. The same pattern repeats in sales discounts, data retention and customer status.

## What exists today

We built **Helix v0**, an internal context engine. It reads exports from all seven systems, extracts facts with dates and sources into a temporal knowledge graph (Graphiti on Neo4j), and serves them through four MCP tools. On top of it runs **Ask Orbisa**, an agent built with the Claude Agent SDK. Helix v0 is in pilot.

| 7 | 67 | 4 | 22 |
|---|---|---|---|
| source systems exported to `data/orbisa/` | ingestion episodes in the temporal graph | MCP tools served by Helix | ground-truth questions in 7 categories |

## What is missing

Ask Orbisa gives answers, but **no one can tell whether an answer is right, why, or what it cost**. There are no traces. We cannot see which facts and sources an answer used. We do not measure correctness. A prompt change can break answers without anyone noticing. Cost and latency per question are printed to a terminal and then lost.

> **THE MISSION**
>
> **Make Helix observable and prove it works.** Instrument Helix and Ask Orbisa with OpenTelemetry and a self-hosted Langfuse. Make every answer traceable to its facts and sources. Measure correctness per category on the 22-question ground truth. Show cost and latency per question. Alert on quality drops. Compare prompt changes before release. Company data stays on our machines.

# 02 Company background



Orbisa sells B2B logistics software to mid-size freight distributors and wholesalers. It was founded in April 2019 in Austin, Texas, by Marina Costa (CEO) and Daniel Okafor (CTO). In October 2026 it has about 420 employees.

### PRODUCTS

- **Orbisa Track**: shipment tracking API and dashboard.
- **Orbisa Bill**: freight billing and audit.

### FUNDING

- Series A in 2021.
- Series B of USD 48M in March 2024.

### PLANS (CONFLUENCE "PLANS AND PRICING")

| Plan | Price | Notes |
|---|---|---|
| **Starter** | USD 500 / month | Annual or monthly |
| **Growth** | USD 2,400 / month | Annual |
| **Enterprise** | Custom, from USD 12,000 / month | Annual, 99.9% monthly uptime SLA (POL-012) |

There is no Platinum plan. "Platinum" was a Brisklane plan name.

## Key customers

| Customer | Plan | ARR (USD) | Contract paper | Renewal | Owner |
|---|---|---|---|---|---|
| **Orion Freight Systems** | Enterprise | 186,000 | Orbisa MSA with clause 7.3 | Renewed 2026-08-15 | Johanna Lindqvist |
| **Kodiak Cold Chain** | Enterprise | 240,000 | Orbisa | Renews 2026-11-30 (at risk) | Johanna Lindqvist |
| **Bluefin Distribution** | Growth | 28,800 | Orbisa standard | Renewed 2026-10-03 | Owen Brandt |
| **Mesa Ridge Foods** | Growth | 28,800 | Orbisa standard | New, started 2026-09-15 | Owen Brandt |
| **Cedar & Pine Wholesale** | Starter | 6,000 (until churn) | Brisklane legacy until 2027-01-31 | Cancelled with full refund 2026-03-06 | Owen Brandt |

## People

Thirteen people appear in the exports, often under different names in different systems (Appendix B). You will meet Tomás Herrera (Head of RevOps), Johanna Lindqvist (Senior Account Executive), Ken Watanabe (Support Lead) and Sam Patel (Support Agent) most often.

## The Brisklane acquisition

On 2025-03-10 Orbisa closed the acquisition of **Brisklane**, a last-mile routing company in Lisbon with about 60 people. Brisklane customers stayed on their Brisklane contracts. Owen Brandt became their Account Executive.

> *Brisklane customers stay on their current Brisklane contracts for now, and Owen Brandt is their Account Executive.*
>
> Marina Costa, #announcements, 2025-03-10 · `slack/announcements/2025-03-10.json`

**WHY THE ACQUISITION SCATTERED KNOWLEDGE**

- **Two rulebooks.** Brisklane's Notion handbook has its own refund rule (full refund within 60 days), retention (12 months) and plan names (Basic, Pro, Platinum).
- **A frozen wiki.** The Notion space became read-only after the acquisition. It was not updated when Orbisa rules changed, and it still says "Status: Active" on old pages.
- **Exceptions in contracts.** Legacy customers like Cedar & Pine keep Brisklane terms until a date set in a signed addendum, stored as a PDF in Drive.
- **Decisions in chat.** New rules are often announced in Slack first. The Confluence page is updated later, or never.

# 03 The business problem

WHAT GOES WRONG

Every customer-facing team asks the same kinds of questions: who approves this, what does the contract say, what did we promise, is this rule still valid. The answer is usually somewhere in seven systems, and often in two places that disagree.

## How teams find answers today

| Team | Typical question | Where they look | What goes wrong |
|---|---|---|---|
| **Support** | Can this customer get a refund? | Zendesk macros, Confluence, memory | The 2023 refund page is still live and looks official. |
| **Sales** | Who approves this discount? | Confluence Discount Approval Matrix, the Sales Director | The matrix is outdated. The newer rule is in a Slack message. |
| **RevOps** | What does this contract allow? | Drive PDFs, HubSpot | Contract clauses override policy, but they sit in PDFs nobody searches. |
| **Finance** | Is this payout approved, and by whom? | Slack, Zendesk internal notes | The Confluence roles page still names Finance as approver. |
| **Customer Success** | What did we promise this customer? | Gong calls, Slack channels, HubSpot | Promises live in call transcripts. One person has three names across systems. |

## The Orion incident

Orion Freight Systems is our anchor case. It shows every failure in one story.

**2025-08-15** — **Orion signs the Orbisa MSA and Order Form OF-2025-0815** — Enterprise, USD 186,000 per year (USD 15,500 per month), auto-renews every 12 months. Signed by Marina Costa and Marcus Ellery. drive/Orion_Freight_Systems_MSA_2025-08-15.pdf

**2026-03-01** — **Refund & Credits Policy v2 (POL-007 v2) takes effect** — Full refund within 14 days of renewal, then service credits only. Contract terms win. The 2023 page is never archived. confluence/Refund-and-Credits-Policy_98309871.html

**2026-03-03** — **Refund approvals move from Finance to RevOps** — Tomás Herrera approves every refund. Finance only pays out. Confluence "Revenue Team — Roles" is not updated. slack/revops/2026-03-03.json

**2026-08-15** (day 0) — **Orion's contract renews** — The 60-day window of clause 7.3 starts.

**2026-08-19** (day 4) — **INC-2291: Tracking API outage, 3 h 12 min** — A bad deploy of tracking-ingest v4.12 caused Kafka consumer lag. Zendesk #48190. confluence/INC-2291-Postmortem_98311400.html

**2026-08-27** day 12 — **INC-2304: Tracking API outage, 1 h 47 min** — An expired TLS certificate on the carrier-webhook gateway. Zendesk #48207. August uptime is 99.33%, below the 99.9% SLA. confluence/INC-2304-Postmortem_98311452.html

**2026-09-08** day 24 — **Q3 QBR call (Gong)** — Johanna promises a service credit if there is another tracking incident this quarter. She does not promise a refund. Dana says their legal team is reviewing the SLA clause. gong/transcripts.json

**2026-09-29** day 45 — **Zendesk #48233: Orion asks for a refund, and gets the stale rule** — Sam Patel answers with the 2023 30-day rule. Dana pushes back. zendesk/comments/48233.json

**2026-09-30** day 46 — **Escalated to RevOps** — Ken Watanabe flags the error. Tomás Herrera wants legal to confirm clause 7.3 before he approves. No refund has been approved yet. slack/cs-orion/2026-09-30.json

> *Thanks Dana. Per our refund policy, annual subscriptions can only be refunded within 30 days of the renewal date, so I'm afraid we can't offer a refund.*
>
> Sam Patel, public reply on Zendesk #48233, 2026-09-29 · `zendesk/comments/48233.json`

> *Their MSA has clause 7.3 (SLA-based refund within 60 days of renewal) and August uptime was 99.33%. I want legal to confirm the clause before I approve anything.*
>
> Tomás Herrera, #cs-orion, 2026-09-30 · `slack/cs-orion/2026-09-30.json`

**WHAT A CORRECT ANSWER NEEDED ON 2026-09-29**

Five facts from five systems: the renewal date (HubSpot, Order Form), clause 7.3 (MSA in Drive), August uptime of 99.33% (Confluence postmortem), the current policy with its "contract terms win" rule (Confluence), and the current approver (Slack). The policy alone says no (14-day window). The contract says yes: the SLA was missed inside the 60-day window, and day 45 is inside it. The approver is Tomás Herrera, not Finance.

## Other symptoms

### KODIAK DISCOUNT APPROVAL

Kodiak Cold Chain (USD 240,000 ARR, renews 2026-11-30) asked for 25% off on a 2026-10-02 call, to match the competitor FleetGlass. On 2026-10-06 in #deal-desk, Rafael answered from the outdated Confluence matrix: "Above 20% is the CFO, ping Priya." Tomás corrected him.

> *Since June 15 it is CEO and CFO for anything above 20% (Marina's note in #sales). The Confluence matrix is outdated, I will fix it.*
>
> Tomás Herrera, #deal-desk, 2026-10-06 · `slack/deal-desk/2026-10-06.json`

### BRISKLANE DATA RETENTION: 12 OR 24 MONTHS?

The Notion page still says 12 months. Nadia Silva decided in #data-platform on 2025-11-04 that the read-only handbook will not be updated. Since 2026-01-01 the rule is 24 months for all customers, including Brisklane customers (Confluence Data Retention Policy v3, 2025-12-15).

> *After 12 months, tracking events are deleted automatically by the nightly cleanup job.*
>
> Notion "Data Retention", Status: Active, last edited November 20, 2024

### CEDAR & PINE STILL "ACTIVE"

Cedar & Pine cancelled and got a full refund, approved by Tomás on 2026-03-06 (Zendesk #48102). HubSpot lists them as a former customer. The read-only Notion customer list still says `Active`.

THE RISK

- **Revenue.** Orion (USD 186,000) is marked "At risk" in HubSpot. Kodiak (USD 240,000) renews on 2026-11-30 and is at risk.
- **Wrong answers in writing.** Sam's reply was a public Zendesk comment to a customer whose legal team is reading the MSA.
- **No audit trail.** When an answer is wrong, nobody can show which page or message it came from. An AI agent that repeats the same mistakes at scale makes this worse, not better.

# 04 Problem analysis

WHY IT IS HARD

The symptoms in section 3 come from four failure modes. Each one breaks a different assumption of keyword search and of plain vector retrieval (RAG). A useful context layer must handle all four.

## 1 Stale facts: the old version is still online

ORBISA EXAMPLE

Confluence "Refund Policy (2023)" (30 days, Finance approves) was never archived. "Revenue Team — Roles" still says Finance approves refunds. The real change on 2026-03-03 lives in a Slack message.

WHY SEARCH AND PLAIN RAG FAIL

**No time.** The old and new pages match the query equally well. A chunk has no "valid from" or "valid until". The ranker cannot prefer the rule that was true on the question's date, and cannot answer "who approved refunds in December 2025?"

## 2 Conflicting sources: two systems disagree

ORBISA EXAMPLE

Retention for Brisklane customers: Notion says 12 months, Confluence v3 says 24 months. Cedar & Pine: Notion says "Active", HubSpot says "Former customer". Orion: the policy says 14 days, the signed MSA says 60 days under clause 7.3.

WHY SEARCH AND PLAIN RAG FAIL

**No provenance.** Chunks lose where they came from and how much weight they carry. A read-only handbook and a signed contract look the same to the model. It may pick one at random, or blend them into a wrong answer.

## 3 Aliases: one person, many names

ORBISA EXAMPLE

Johanna Lindqvist is "Jo L." (`jo.l`) in Slack, "Jo Lindqvist" in Zendesk, "Johanna Lindqvist" in HubSpot and Gong, and "Jo (Orbisa sales)" in the Notion team directory. The QBR promise is in Gong; the recap is from "Jo L." in Slack.

WHY SEARCH AND PLAIN RAG FAIL

**No entity resolution.** Keyword search for "Johanna" misses "Jo L.". Vectors for short names are noisy. The system cannot join "what Jo L. wrote" with "what Johanna said on the call" into one person and one account owner.

### 4 Missing facts and false premises

**ORBISA EXAMPLE**

"What is the refund window for the Platinum plan?" There is no Orbisa Platinum plan; it was a Brisklane plan. "When was Orion's refund approved?" It was not. "What did the 2025 SOC 2 audit conclude?" Nothing in our sources covers it.

**WHY SEARCH AND PLAIN RAG FAIL**

**Top-k always returns something.** Retrieval returns the nearest chunks even when the answer is not there, for example Brisklane's 60-day rule for "Platinum". The model then answers with confidence. Saying "not found" needs evidence about what is *not* known.

### Why context must be served through one interface

Orbisa will not have one agent. Support wants a Zendesk assistant, and engineers already use Claude Code, Codex and Cursor. If each tool builds its own retrieval, each has its own bugs and its own stale copy of the rules. So context must be **one service with one interface**: the **Model Context Protocol (MCP)**. An MCP server publishes typed, described tools that any MCP client can call. Fix a fact in Helix once, and every agent gets the fix.

**KEY INSIGHT**

The hard part is not finding text. It is knowing **which fact was true on a given date**, **which source it came from**, and **who a name refers to**. Keyword search and plain vector RAG have no time, no provenance and no entity resolution, and top-k always returns something. That is why Helix stores facts with time and provenance, not just chunks.

## 05 Requirements

FR · NFR

Requirements come from section 4 and from the stakeholders on the cover. The last column shows Helix v0 today.

Met built and working Partial built but incomplete or unmeasured Not met missing Target a goal for this mission

### Functional requirements

| ID | Requirement | Helix v0 |
|---|---|---|
| FR-1 | Answer **as of a date**. Hide facts recorded after that date. Mark facts that were no longer true. | Met `as_of` on `search_context` |
| FR-2 | **Cite sources**. Every answer lists the documents it relied on, by path. | Partial cited, not checked |
| FR-3 | **Resolve aliases** of people, customers and policies. | Partial `find_entities`, unmeasured |
| FR-4 | **Prefer the authoritative fact**: a signed contract over a general policy, a valid fact over a superseded one. Show the conflict. | Partial prompt rule only |
| FR-5 | **Say "not found"** and reject false premises. Never invent names, dates, amounts or approvals. | Partial prompt rule only |
| FR-6 | **Serve any agent through MCP**, with the same tools and descriptions. | Met 4 tools over streamable HTTP |
| FR-7 | **Verify against the original document** before quoting numbers or wording. | Met `read_source` |

### Non-functional requirements

| ID | Requirement | Helix v0 |
|---|---|---|
| NFR-1 | **Provenance and audit trail.** For every answer: who asked, the as-of date, the tool calls, the facts and sources used, the answer. | Not met facts have provenance; runs leave no record |
| NFR-2 | **Data residency.** Company data stays in-house, because the context holds customer contracts and names. Graph, embeddings and observability run on our machines. | Partial local graph and embeddings; no observability yet |
| NFR-3 | **Cost.** Cost per question and per model call is visible and can be compared. | Not met printed to terminal only |
| NFR-4 | **Latency.** Time per question and per tool call is visible. Support will not wait minutes for an answer. | Not met printed to terminal only |
| NFR-5 | **Open standards.** MCP for context, OpenTelemetry for telemetry, open-source components. | Partial MCP yes, OpenTelemetry not yet |
| NFR-6 | **Model choice.** Models can be changed by configuration. | Met `HELIX_*_MODEL`, OpenRouter option |
| NFR-7 | **Measurable quality.** Correctness is scored on the ground truth, and regressions are caught before release. | Not met |
| NFR-8 | **Resilience.** Observability is a layer, not a dependency. Answers keep working when it is down. | Target |

# 06 Solution overview

HELIX V0 + ASK ORBISA

The solution has two parts. **Helix** is the context engine: it knows the facts. **Ask Orbisa** is the agent: it asks Helix and writes the answer. They talk only through MCP.

**HELIX — CONTEXT ENGINE**

`src/context/`

- One parser per system turns raw exports into **episodes** (67 in total).
- Graphiti uses Claude Haiku 4.5 to extract **entities** and **facts** from each episode, using our ontology of 8 types.
- Facts are stored in Neo4j with validity dates and links to their source episodes.
- An MCP server exposes 4 tools on `http://127.0.0.1:8765/mcp`.

**ASK ORBISA — AGENT**

`src/clients/agent/`

- Built on the Claude Agent SDK with Claude Opus 4.5.
- Its only tools are the Helix MCP tools (`mcp__helix__*`).
- System prompt in `system_prompt.md`: use the as-of date, search from several angles, prefer contracts over policy, verify in the source, say so when the answer is missing.
- Answer format: direct answer, short reasoning bullets, a "Sources:" line.

### How one question flows

1. An employee asks a question with an **as-of date**. The runner (`src/clients/agent/runner.py`) sends `As-of date: … Question: …` to the agent, which plans its searches: customer, contract, policy, approver.

2. `search_context(query, as_of, limit)`: Helix runs a hybrid search on the graph, attaches the source episodes, labels each fact's status, and hides facts recorded after `as_of`. The agent calls it several times, from different angles.

3. `find_entities(query, limit)`: looks up a name such as "Jo L." or "Orion" and returns entity summaries. Whether aliases were merged into one entity at ingestion has not been checked.

4. `read_source(path)`: returns the original text (up to 15,000 characters) behind a key fact, such as a contract clause. `list_sources()` lists every valid path by system.

5. The agent weighs the facts (valid on the date, contract over policy, newest decision over older page) and answers with reasoning and a **Sources:** line. The SDK reports turns, cost and duration, which today are only printed to the terminal.

# 07 Architecture

COMPONENTS · SEQUENCE · TIME

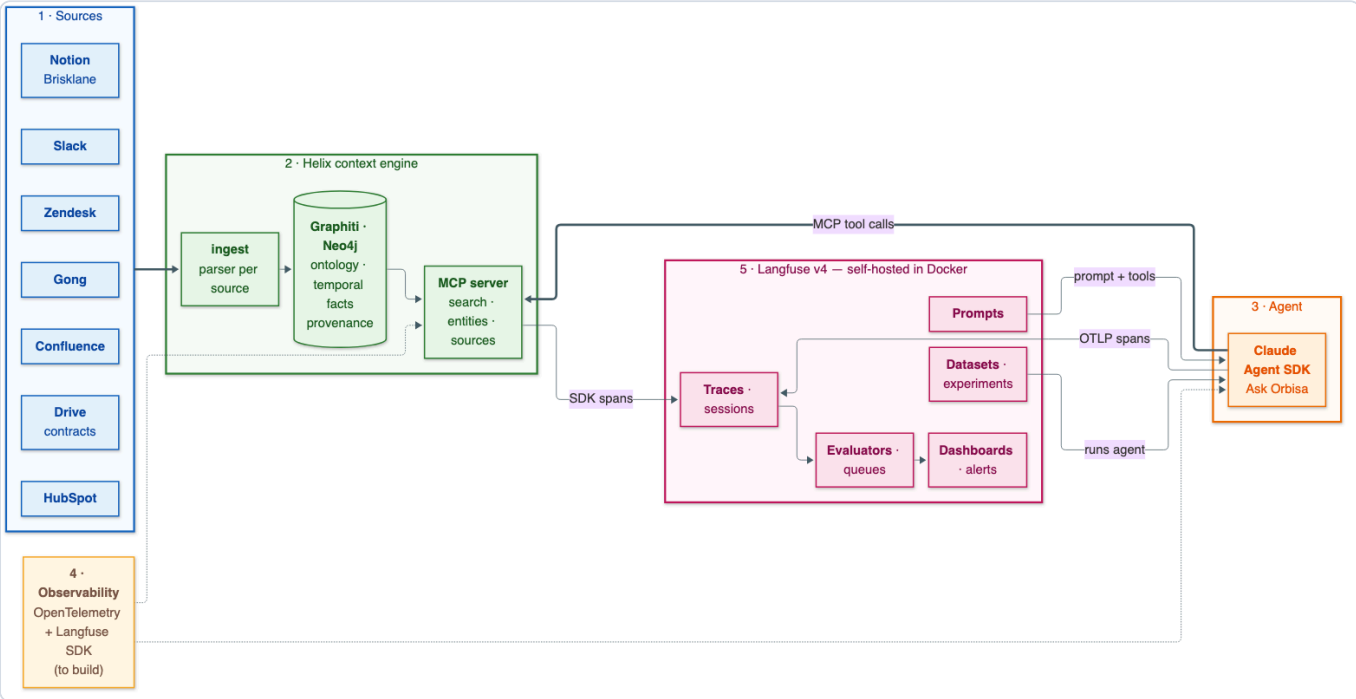



**Figure 1.** Target architecture. Boxes 1–3 exist today (Helix v0). Boxes 4–5 are the observability layer this team will build: OpenTelemetry and the Langfuse SDK, sending to Langfuse v4 self-hosted in Docker Compose (`infra/docker-compose.yml`) next to Helix and Neo4j.

## Components

| Component | File path | Responsibility |
|---|---|---|
| **Source parsers** | `src/context/ingest/sources.py` | One parser per system. Turns exports into dated episodes with a path, a description and a content digest. |
| **Ingestion job** | `src/context/ingest/__main__.py` | Sends new or changed episodes to Graphiti. Keeps a manifest in `.helix/ingested.json`. Supports `--dry-run`, `--reset`, `--system`. |
| **Ontology** | `src/context/ontology/__init__.py` | 8 entity types as Pydantic models: Customer, Person, Team, Policy, Contract, Plan, Incident, Product. |

| Component | File path | Responsibility |
|---|---|---|
| **Graph client** | `src/context/graph.py` | Builds Graphiti with the Anthropic LLM client, the local embedder and the local reranker. |
| **Local models** | `src/context/embedding.py` | fastembed embedding (BAAI/bge-small-en-v1.5) and cross-encoder reranker (Xenova/ms-marco-MiniLM-L-6-v2) on CPU. |
| **Retrieval** | `src/context/retrieval/__init__.py` | Temporal fact search, fact status, entity lookup, source reading, source listing. |
| **MCP server** | `src/context/mcp/server.py` | Defines the 4 tools and server instructions. Streamable HTTP on port 8765. |
| **Agent runner** | `src/clients/agent/runner.py` | Claude Agent SDK options, MCP connection, run loop. Collects answer, tool calls, turns, cost, duration. |
| **System prompt** | `src/clients/agent/system_prompt.md` | Agent behaviour and answer format. Lives in a file. |

## Request sequence for Q01

Q01 asks, as of 2026-09-29: "*Orion Freight Systems is asking for a refund 45 days after their renewal. Are they eligible, and who must approve it?*" A typical run looks like this. The exact calls are chosen by the model and differ between runs, which is one reason we need traces.

| # | From → to | What happens |
|---|---|---|
| 1 | CLI → Agent SDK | `uv run python -m clients.agent --id Q01` loads Q01 from `questions.jsonl` and calls `query()` with the system prompt, `claude-opus-5-5`, `max_turns=15`, only `mcp__helix__*` tools. |
| 2 | agent → Helix | `search_context("Orion refund rights after renewal", as_of="2026-09-29")` |
| 3 | Helix → Neo4j | Graphiti hybrid search over facts. Retrieval loads the source episodes, computes each fact's status and hides facts recorded after 2026-09-29. |
| 4 | agent → Helix | `search_context("who approves refunds", as_of=…)` returns the RevOps rule from `slack/revops/2026-03-03.json` as *true on as_of*, and the Finance rule as *no longer true*. |
| 5 | agent → Helix | `search_context("August 2026 uptime SLA")` finds 99.33% in the INC-2304 postmortem. |
| 6 | agent → Helix | `read_source("drive/Orion_Freight_Systems_MSA_2025-08-15.pdf")` to read the exact wording of clause 7.3. |
| 7 | agent → user | Answer: eligibility under clause 7.3, approver Tomás Herrera, reasoning bullets, **Sources:** line. |
| 8 | SDK → runner | `ResultMessage` with `num_turns`, `total_cost_usd`, `duration_ms`. Printed, then lost. |

## The temporal model

Graphiti stores each fact as an edge between two entities, for example *RevOps — approves — refunds*. Helix uses two kinds of time for each fact:

- **Valid time**: when the fact is true in the world. Exposed as `valid_from` and `valid_until` (Graphiti's `valid_at` and `invalid_at`). When a newer episode contradicts a fact, the extraction model sets `valid_until`; nobody has checked that it closes the right facts. Old facts are never deleted.
- **Recorded time**: when Orbisa wrote it down. This is the date of the source episode, for example the day of the Slack message or the "last modified" date of a Confluence page.

With an `as_of` date, `fact_status()` in `src/context/retrieval/__init__.py` works like this:

**`true on as_of`**
Valid on that date and recorded on or before it. The agent should use it.

**`no longer true on as_of`**
`valid_until` is on or before the date. Useful as history: "this used to be Finance".

**hidden**
Recorded after `as_of`, or not yet valid. Removed from the result; only `hidden_as_future` counts them.

Without an `as_of` date, facts are labelled `current` or `superseded`. Hiding later facts is what makes "Who approved refunds on 2025-12-01?" answerable: the 2026-03-03 Slack message did not exist yet.

## 08 Technology stack

VERSIONS FROM UV.LOCK AND COMPOSE

Versions below are pinned in `pyproject.toml` and resolved in `uv.lock`, or set as image tags in `infra/docker-compose.yml`. Section 9 explains why each choice was made.

| Layer | Choice | Version | Role |
|---|---|---|---|
| **Language & packaging** | Python + uv | Python 3.12 ( `>=3.12, <3.13` ), uv 0.9 in the image | All code; reproducible installs from `uv.lock`. |
| **Graph database** | Neo4j Community + APOC | `neo4j:5.26-community` ; Python driver 6.4.1 | Stores entities, facts and episodes. Browser on port 7474. |
| **Knowledge engine** | graphiti-core | 0.31.1 | Episode ingestion, entity and fact extraction, bi-temporal edges, hybrid search. |
| **Embeddings** | fastembed (ONNX, CPU) | fastembed 0.9.0, onnxruntime 1.31.0 | `BAAI/bge-small-en-v1.5` , 384 dimensions, runs locally. |
| **Reranking** | fastembed cross-encoder | fastembed 0.9.0 | `Xenova/ms-marco-MiniLM-L-6-v2` , runs locally. Replaces Graphiti's default hosted reranker. |
| **Extraction model** | Claude Haiku 5.5 | `claude-haiku-5-5` ; anthropic SDK 0.125.0 | Graphiti's LLM for entity and fact extraction at ingestion. |
| **Answer model** | Claude Opus 5.5 | `claude-opus-5-5` | Ask Orbisa's reasoning and answers. |
| **Context interface** | MCP Python SDK | mcp 2.3.0, streamable HTTP | Helix MCP server on `:8765/mcp` . |
| **Agent runtime** | Claude Agent SDK | claude-agent-sdk 0.2.168 (0.2.x) | Agent loop, MCP client, cost and turn reporting. |
| **Parsing** | pypdf, Beautiful Soup | pypdf 6.20.1, beautifulsoup4 4.15.0 | Contract PDFs and Confluence HTML. |
| **Runtime** | Docker Compose | project `orbisa` ; image `python:3.12-slim` | Neo4j, helix-mcp, ingest and agent jobs, Langfuse stack. |
| **Observability** not built | Langfuse, self-hosted | `langfuse:4` and `langfuse-worker:4` ; Python SDK 4.18.0 (locked, unused) | Traces, prompts, datasets, experiments, evaluators, dashboards. Backed by `postgres:17` , `clickhouse-server:25.12` , `redis:7` and MinIO. |

| Layer | Choice | Version | Role |
|---|---|---|---|
| **Telemetry standard** not built | OpenTelemetry | opentelemetry-sdk 1.45.1 (in lock) | Span format and OTLP export from the agent and Helix. |

# 09 Tool selection

DECISION RECORDS

Each card records one decision: context, options, decision and trade-offs. Challenge them as a team.

## 9a Knowledge engine

**Decision: Graphiti 0.31.1**

| | |
|---|---|
| CONTEXT | Facts change over time, sources conflict, and every answer must cite its source. The corpus is business documents, not code, and it grows every day. |
| OPTIONS | **Graphiti** (temporal knowledge graph) · **Microsoft GraphRAG**: suits static corpora and global summaries, but indexing is expensive · **LightRAG**: cheaper, but no time model · **Graphify**: maps code repositories, not business documents · **plain vector database**: no time, no entities, no conflicts. |
| DECISION | Graphiti. Bi-temporal facts (valid time and recorded time). Provenance from every fact to its source episodes. Custom ontology as Pydantic types ( `src/context/ontology/` ). Incremental updates with `add_episode` , no full re-index. |
| TRADE-OFFS | Needs a graph database. Every episode costs LLM calls at ingestion (cost and rate limits; see `SEMAPHORE_LIMIT` ). Extraction is non-deterministic: a wrong fact at ingestion becomes a wrong answer later, and today we cannot see it. |

## 9b Graph store

**Decision: Neo4j 5.26 Community**

| | |
|---|---|
| CONTEXT | Graphiti needs a graph store with full-text and vector search. Engineers must be able to look at the graph. |
| OPTIONS | **Neo4j** · **FalkorDB** (Redis-based, supported by Graphiti) · **Kuzu** (embedded, no server). |
| DECISION | Neo4j: mature, Graphiti's default backend, and Neo4j Browser ( `http://localhost:7474` ) lets engineers explore entities and facts with Cypher. |
| TRADE-OFFS | A JVM service to run (1G heap in compose). No clustering in Community edition; fine for a pilot. |

## 9c Embeddings and reranking

**Decision: local fastembed**

| | |
|---|---|
| CONTEXT | Every episode and every query is embedded. The text includes contracts and customer names. |
| OPTIONS | **Hosted embedding and rerank APIs** (Graphiti's defaults) · **local ONNX models with fastembed**. |
| DECISION | Local: `BAAI/bge-small-en-v1.5` embeddings and `Xenova/ms-marco-MiniLM-L-6-v2` reranker. No extra vendor key, and the text stays on the machine. Models are baked into the Docker image, so containers start offline. |
| TRADE-OFFS | Weaker quality than large hosted models, English only, CPU time. When an answer is wrong, retrieval quality is a suspect, and we need traces to tell. |

## 9d Interface to agents

**Decision: MCP**

| | |
|---|---|
| CONTEXT | Many agents need the same context (section 4). Different teams build them with different tools. |
| OPTIONS | **Custom REST API** (every client writes its own tool wrappers) · **MCP**. |
| DECISION | MCP Python SDK 2.3 over streamable HTTP. Any MCP client (Claude Agent SDK, Claude Code, Codex, Cursor) uses the same tools, descriptions and server instructions, so all agents share one context. |
| TRADE-OFFS | A young protocol that still changes. Tool descriptions are prompts too, so they need versions. Tracing across the agent → MCP boundary needs trace-context propagation. |

## 9e Agent runtime

**Decision: Claude Agent SDK 0.2.x**

| | |
|---|---|
| CONTEXT | The agent runs a loop: plan, call tools, read results, answer. It must speak MCP and report what it did. |
| OPTIONS | **Hand-written tool loop** on the Messages API · **a graph framework** (explicit state machine) · **Claude Agent SDK**. |
| DECISION | Agent SDK: built-in agent loop, native MCP configuration, built-in OpenTelemetry export, and cost, turns and duration in every `ResultMessage`. |
| TRADE-OFFS | Less control over each step. Tied to Claude-compatible endpoints (OpenRouter works through `ANTHROPIC_BASE_URL`). Runs the bundled Claude Code CLI as a subprocess. |

## 9f Models

**Decision: Haiku 5.5 + Opus 5.5**

| | |
|---|---|
| CONTEXT | Extraction runs on every episode. Answers need careful reasoning across conflicting, dated facts. |
| OPTIONS | One model for everything · **a cheap model for bulk extraction and a strong model for answers**. |
| DECISION | Claude Haiku 5.5 ( `claude-haiku-5-5` ) extracts facts from every episode. Claude Opus 5.5 ( `claude-opus-5-5` ) reasons across conflicting, dated facts. Both set in `.env`. OpenRouter is an optional route for the agent ( `HELIX_AGENT_PROVIDER=openrouter` ). |
| TRADE-OFFS | Errors from the cheaper model show up later as wrong facts. The cost of an Opus answer is unknown today. Comparing models needs experiments on the ground truth. |

## 9g Observability platform

**Decision: Langfuse v4 + OpenTelemetry**

| | |
|---|---|
| CONTEXT | We need traces, prompt versions, datasets, experiments, evaluators, human review and alerts, on data that must stay in-house. |
| OPTIONS | **Langfuse** · **LangSmith** (strongest with LangChain; self-hosting is an enterprise offer) · **Arize Phoenix** (open source, OpenTelemetry-based, strong tracing and evals) · **Braintrust** (strong evals and experiments; SaaS first). |
| DECISION | Langfuse: open source (MIT core), self-hostable (data residency), OpenTelemetry-native, and tracing, prompt management, datasets, experiments, evaluators and annotation queues in one place. OpenTelemetry is the standard underneath, so spans are not locked to one vendor. |
| TRADE-OFFS | Four extra stores to run (Postgres, ClickHouse, Redis, MinIO); we own upgrades and backups. Some governance features need an Enterprise license (9h). |

### 9h Deployment of Langfuse

**Decision: self-hosted in Docker Compose**

| | |
|---|---|
| CONTEXT | Orbisa's context includes customer data: contract terms, ARR, names, support threads. Traces will contain all of it. |
| OPTIONS | **Langfuse Cloud** (managed) · **self-hosted** next to Helix and Neo4j. |
| DECISION | Self-hosted open-source edition, no license key, defined in `infra/docker-compose.yml` (org "Orbisa", project "Ask Orbisa"). What this means: |

| Availability in our setup | Features |
|---|---|
| Free, self-hosted | Tracing · OpenTelemetry ingestion · prompt management · playground · datasets · experiments · LLM-as-a-judge (needs an LLM connection) · annotation queues · dashboards · alerts (v4) · Metrics API · built-in MCP server and CLI · code evaluators (TypeScript/JavaScript only, with the local dispatcher). |
| Free, in the SDK | Client-side masking in the Langfuse SDK (mask data before it leaves the process). |
| Enterprise key — not used | Server-side data masking · audit logs · data retention policies · project-level RBAC · protected prompt labels · SCIM and organization management API · UI customization. |
| Beta — not used | Langfuse Assistant: public beta for self-hosted from v4.28; needs an instance-level AI provider configuration. Not used in this project. |
| Cloud only | Spend alerts · vendor support and SLA. |

**WHAT THIS MEANS FOR THE TEAM**

Everything the mission needs is in the free self-hosted edition. Masking must happen in our code with the SDK, not on the server. There is no server audit log, so the trace itself is our audit record. There are no protected prompt labels, so we agree a label convention (for example, only RevOps-reviewed prompts get `production`).

# 10 Current state (brownfield)

WHAT YOU INHERIT

You are not starting from zero. Helix v0 and Ask Orbisa run end to end. What is missing is the ability to see inside them and to measure them.

## What exists, and where

| Asset | Path | State |
|---|---|---|
| **7 source exports** | `data/orbisa/` (one folder per system) | Real export formats (Appendix A). |
| **67 ingestion episodes** | `src/context/ingest/` | Confluence 14, Notion 8, Slack 19, Drive 6, Zendesk 12, HubSpot 5, Gong 3. |
| **Ontology** | `src/context/ontology/__init__.py` | 8 entity types with typed attributes. |
| **Temporal graph** | Neo4j, group `orbisa` | Built by `python -m context.ingest`. |
| **4 MCP tools** | `src/context/mcp/server.py` | `search_context`, `find_entities`, `read_source`, `list_sources`. |
| **Ask Orbisa agent** | `src/clients/agent/` | `python -m clients.agent --id Q01` |
| **Ground truth** | `data/orbisa/truth/questions.jsonl` | 22 questions in 7 categories (Appendix C). |

| Asset | Path | State |
|---|---|---|
| **Tests** | `tests/` | Parsers, truth file format, retrieval logic. No answer-quality tests. |
| **Observability package** | `src/observability/` | Empty sub-packages, waiting for this mission. |
| **Infrastructure** | `infra/docker-compose.yml` | Neo4j, Helix and Langfuse v4 defined; Langfuse not wired to code. |

### What works

All seven systems parse into dated episodes, and re-ingestion only sends changed ones. Facts carry validity dates and source paths, and `as_of` hides later facts. Any MCP client can use Helix, and the agent answers with reasoning and a Sources line. Models, the MCP URL and the provider are configurable.

### Known gaps

| Gap | Consequence today |
|---|---|
| **No traces** | We cannot replay what the agent did for a question, step by step. |
| **No view of facts used** | We cannot see which facts and sources an answer relied on, or whether Helix even returned the right ones. |
| **No correctness measurement** | We do not know how often Ask Orbisa is right, overall or per category. |
| **No regression protection** | A change to `system_prompt.md` or a tool description can break answers silently. |
| **Unknown cost and latency** | `AgentRun` collects `cost_usd` and `duration_ms`, prints them, and drops them. Ingestion cost is not recorded at all. |
| **No monitoring or alerts** | A drop in quality is found by a customer, as with Orion. |
| **Prompts live in files** | No versions, no labels, no link between a prompt version and the answers it produced. |
| **Ground truth not wired** | `questions.jsonl` is only read by the CLI to print the expected answer next to the real one. |

## 11 The mission

FOR THE PLATFORM & AI ENGINEERING TEAM

MISSION

**Make Helix observable and prove it works.**

### Questions the team must answer

**Q1** For any answer: which tool calls ran, which facts and sources came back, and were they true on the as-of date?

**Q2** When an answer is wrong, where did it break: the source data, extraction, retrieval, or the agent's reasoning?

**Q3** How correct is Ask Orbisa per category: anchor, temporal, conflict, multi-source, alias, false premise, lookup?

| | |
|---|---|
| **Q4** | What does one question cost, how long does it take, and which step dominates? |
| **Q5** | Does a change to the system prompt, a tool description or a model make answers better or worse, before we release it? |
| **Q6** | How do we learn that quality dropped, before a customer tells us? |
| **Q7** | How do RevOps and Support experts review real answers and feed corrections back? |

## Success criteria

| Criterion | How we will check it |
|---|---|
| **Every answer is traceable** | One trace per question links the agent turns, each MCP tool call, the facts returned, their source paths and the final answer. |
| **Correctness is measured per category** | All 22 ground-truth questions run as a dataset. Scores are reported per category, not only as one average. |
| **Cost and latency are visible** | Cost, tokens and duration per question and per step, on a dashboard. |
| **A quality drop raises an alert** | Scores on live answers are monitored; a drop below an agreed threshold notifies the team. |
| **Prompt changes are compared before release** | The system prompt is versioned. A new version runs as an experiment against the ground truth and is compared with the current one. |

**CONSTRAINTS**

- **Self-hosted only.** Langfuse runs in our Docker Compose stack. No Langfuse Cloud.
- **The agent keeps working when Langfuse is down.** Observability is a layer, not a dependency. Missing keys or an unreachable server must not fail a question.
- **No customer data leaves the laptop** through observability. Traces, prompts, datasets and scores stay local. The model calls to Anthropic, as in Helix v0, are the only external flow.

**OUT OF SCOPE**

- Rebuilding Helix: the graph, the ontology and the retrieval logic stay as they are, unless a measurement shows a defect.
- Adding new sources or systems.
- Changing the ground truth to make scores look better.
- Enterprise-licensed Langfuse features (see 9h).

**WHERE YOUR CODE GOES**

Put the new code in `src/observability/`, in the existing sub-packages. Keep `src/context/` and `src/clients/` changes small: a decorator, a hook, a context header. The Langfuse stack is already defined in `infra/docker-compose.yml`; `scripts/init_env.py` generates its secrets.

Daniel Okafor, CTO · Office of the CTO, Orbisa · October 2026

# A Systems inventory

APPENDIX

| System | Folder | Export format | What lives there | Known issues | Ep. |
|---|---|---|---|---|---|
| **Slack** | `slack/` | Workspace export: users, channels, one JSON file per channel per day | Decisions, announcements, incidents, deal desk, account channels | Rules change here first and never reach the wiki. Handles like `jo.l`. | 19 |
| **Confluence** | `confluence/` | Space HTML export, one file per page | Policies (POL-007, POL-012), plans, discount matrix, team roles, postmortems | Refund Policy (2023) never archived. Roles page and Discount matrix outdated. | 14 |
| **Notion** | `notion/` | Markdown and CSV; page ids of 32 hex characters | Brisklane handbook: refunds, retention, team, customers | Read-only since the acquisition. 12-month retention, Cedar & Pine "Active", Platinum plan. | 8 |
| **Google Drive** | `drive/` | Signed contract PDFs | Orion MSA and Order Form, Kodiak and Bluefin Order Forms, Cedar & Pine addendum | Contract terms override policy, but the text is only in PDFs. | 6 |
| **HubSpot** | `hubspot/` | CRM CSV exports: companies, deals, contacts | Owner, plan, ARR, renewal dates, lifecycle, health | Names differ from Slack handles. Contract details are summaries. | 5 |
| **Zendesk** | `zendesk/` | Ticket export JSON, users, comments per ticket | Tickets, public replies, internal notes | Agents answer from memory or macros. Public replies can quote stale rules. | 12 |
| **Gong** | `gong/` | `calls.json` and `transcripts.json` (API format) | QBR and renewal calls | Promises exist only in transcripts, by speaker id. | 3 |

Folders are relative to `data/orbisa/`. "Ep." is the number of ingestion episodes per system (67 in total).

# B People and customers

APPENDIX

| Person | Role | Aliases in the systems |
|---|---|---|
| **Marina Costa** | CEO, co-founder | Slack `marina` |
| **Daniel Okafor** | CTO, co-founder | Slack `daniel.o` |
| **Priya Raman** | CFO | Slack `priya` |
| **Tomás Herrera** | Head of RevOps | Slack `tomas.h` |
| **Rafael Mendes** | Sales Director | Slack `rafa` |
| **Johanna Lindqvist** | Senior Account Executive | Slack `jo.l` (display "Jo L."), HubSpot and Gong "Johanna Lindqvist", Zendesk "Jo Lindqvist" |
| **Owen Brandt** | Account Executive | Slack `owen` |
| **Aisha Bello** | VP Customer Success | Slack `aisha` |
| **Ken Watanabe** | Support Lead | Slack `ken.w`, Zendesk "Ken Watanabe" |
| **Sam Patel** | Support Agent | Zendesk "Sam Patel" |
| **Nadia Silva** | Data Platform Lead | Slack `nadia` |
| **Ingrid Holm** | Site Reliability Engineer | Slack `ingrid` |
| **Luca Ferretti** | Brisklane founder, now Director of Routing Product | Slack `luca`, Notion "Luca" |

**Customer contacts:** Dana Whitfield (Orion, VP Operations), Marcus Ellery (Orion, Procurement), Hal Brennan (Kodiak, COO), Sofia Ruiz (Bluefin, Finance Manager), Tom Avery (Cedar & Pine, Owner). Open items in October 2026: Orion refund request #48233 (escalated to RevOps, not approved), Kodiak 25% discount request (needs CEO and CFO), Bluefin refund request #48251 (escalated, not approved).

# C Ground-truth overview

APPENDIX

The file `data/orbisa/truth/questions.jsonl` holds 22 questions. Each item has `id`, `category`, `as_of`, `question`, `expected_answer`, `expected_facts`, `expected_sources` and `trap` (the typical wrong answer). The expected answers are deliberately not printed here. The set is small, so report scores per category and do not tune prompts on a single question.

| Category | Count | What it tests | Example question (as of) |
|---|---|---|---|
| **anchor** | 1 | The full Orion case across five systems | Q01 (2026-09-29): Orion Freight Systems is asking for a refund 45 days after their renewal. Are they eligible, and who must approve it? |
| **temporal** | 5 | The same question has different answers on different dates | Q03 (2025-12-01): Who approves customer refunds at Orbisa? |
| **conflict** | 3 | Two sources disagree; pick the valid one | Q08 (2026-10-15): How long does Orbisa keep shipment tracking data for customers that came from Brisklane? |
| **multi_source** | 5 | Combine facts from several systems | Q05 (2026-09-15): What service credit is Orion entitled to for August 2026 under the SLA policy? |
| **alias** | 1 | Resolve one person across systems | Q06 (2026-10-15): Who owns the Orion account, and what did they promise Orion on the September 2026 QBR call? |
| **false_premise** | 3 | The question assumes something false or unknown | Q13 (2026-10-15): What is the refund window for Orbisa's Platinum plan? |
| **lookup** | 4 | A single fact from one or two documents | Q17 (2026-10-15): What caused INC-2291? |
| **Total** | 22 | IDs: temporal Q02, Q03, Q07, Q09, Q21 · conflict Q04, Q08, Q12 · multi_source Q05, Q10, Q11, Q18, Q20 · false_premise Q13, Q14, Q15 · lookup Q16, Q17, Q19, Q22 | |

# D Glossary

APPENDIX

**Annotation queue**
A Langfuse list of traces for humans to review and score.

**`as_of`**
The date a question is asked. Helix hides facts recorded after it.

**Bi-temporal**
Tracking two times per fact: when it is true (valid time) and when it was recorded.

**Dataset / experiment**
In Langfuse: a set of inputs with expected outputs; an experiment runs the system on it and scores the results.

**Entity**
A thing in the graph: customer, person, team, policy, contract, plan, incident or product.

**Episode**
One unit of source content sent to Graphiti, such as a Slack day, a wiki page or a ticket thread. It has a date and a path.

**Evaluator / LLM-as-a-judge**
Code or a model prompt that scores an answer, for example for correctness.

**Fact**
An edge between two entities with a sentence, validity dates and source episodes.

**MCP**
Model Context Protocol. A standard for serving tools and context to any AI client.

**OpenTelemetry (OTel), OTLP**
The open standard for traces and spans, and its wire protocol.

**Trace / span / session**
One request end to end / one step inside it / a group of related traces.